

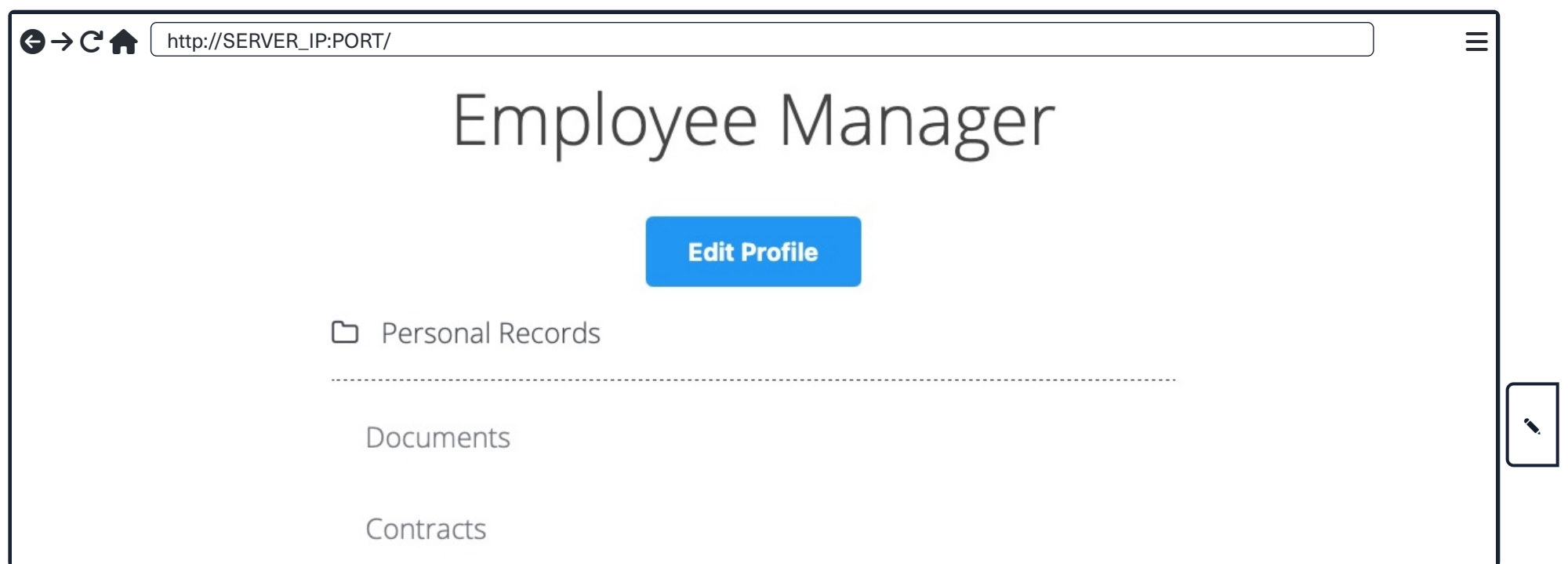
1. IDOR in Insecure APIs

So far, we have only been using IDOR vulnerabilities to access files and resources that are out of our user's access. However, IDOR vulnerabilities may also exist in function calls and APIs, and exploiting them would allow us to perform various actions as other users.

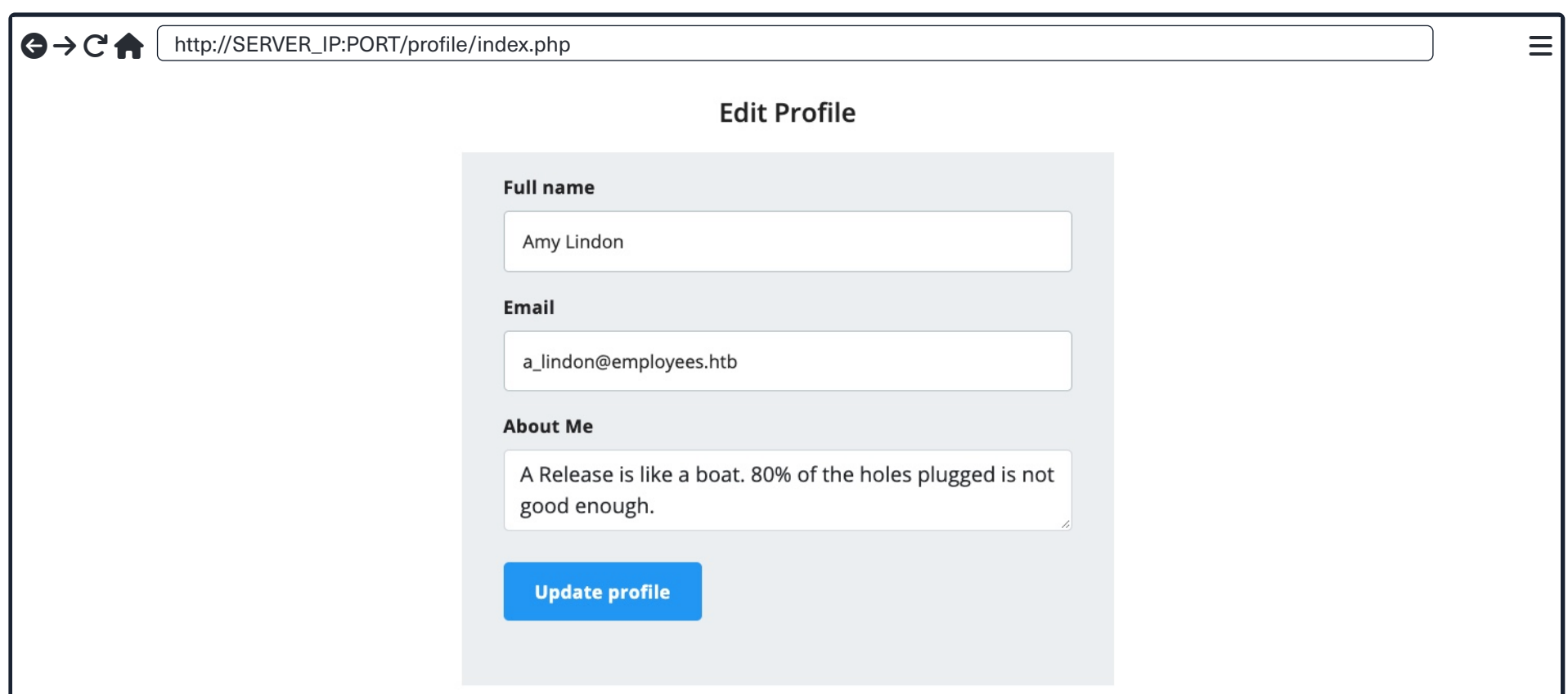
While **IDOR Information Disclosure Vulnerabilities** allow us to read various types of resources, **IDOR Insecure Function Calls** enable us to call APIs or execute functions as another user. Such functions and APIs can be used to change another user's private information, reset another user's password, or even buy items using another user's payment information. In many cases, we may be obtaining certain information through an information disclosure IDOR vulnerability and then using this information with IDOR insecure function call vulnerabilities, as we will see later in the module.

2. Identifying Insecure APIs

Going back to our **Employee Manager** web application, we can start testing the **Edit Profile** page for IDOR vulnerabilities:



When we click on the **Edit Profile** button, we are taken to a page to edit information of our user profile, namely **Full Name**, **Email**, and **About Me**, which is a common feature in many web applications:



We can change any of the details in our profile and click **Update profile**, and we'll see that they get updated and persist through

```
[eu-academy-3]-[10.10.14.104]-[htb-ac-819475@htb-mew08x7bnw]-[~]  
[★$]  
Forward Drop Intercept is on Action Open Browser  
Pretty Raw Hex \n ⋮  
1 PUT /profile/api.php/profile/1 HTTP/1.1  
2 Host: 178.128.160.242:32504  
3 Content-Length: 208  
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/92.0.4515.159 Safari/537.36  
5 Content-type: application/json  
6 Accept: */*  
7 Origin: http://178.128.160.242:32504  
8 Referer: http://178.128.160.242:32504/profile/index.php  
9 Accept-Encoding: gzip, deflate  
10 Accept-Language: en-US,en;q=0.9  
11 Cookie: role=employee  
12 Connection: close  
13  
14 {  
  "uid":1,  
  "uuid":"40f5888b67c748df7efba008e7c2f9d2",  
  "role":"employee",  
  "full_name":"Amy Lindon",  
  "email":"a_lindon@employees.htb",  
  "about":"A Release is like a boat. 80% of the holes plugged is not good enough."  
}
```

We see that the page is sending a **PUT** request to the `/profile/api.php/profile/1` API endpoint. **PUT** requests are usually used in APIs to update item details, while **POST** is used to create new items, **DELETE** to delete items, and **GET** to retrieve item details. So, a **PUT** request for the **Update profile** function is expected. The interesting bit is the JSON parameters it is sending:

Code: **json**

```
{  
  "uid": 1,  
  "uuid": "40f5888b67c748df7efba008e7c2f9d2",  
  "role": "employee",  
  "full_name": "Amy Lindon",  
  "email": "a_lindon@employees.htb",  
  "about": "A Release is like a boat. 80% of the holes plugged is not good enough."  
}
```

We see that the **PUT** request includes a few hidden parameters, like **uid**, **uuid**, and most interestingly **role**, which is set to **employee**. The web application also appears to be setting the user access privileges (e.g. **role**) on the client-side, in the form of our **Cookie: role=employee** cookie, which appears to reflect the **role** specified for our user. This is a common security issue. The access control privileges are sent as part of the client's HTTP request, either as a cookie or as part of the JSON request, leaving it under the client's control, which could be manipulated to gain more privileges.

So, unless the web application has a solid access control system on the back-end, **we should be able to set an arbitrary role for our user, which may grant us more privileges**. However, how would we know what other roles exist?

2. Exploiting Insecure APIs

We know that we can change the **full_name**, **email**, and **about** parameters, as these are the ones under our control in the HTML form in the `/profile` web page. So, let's try to manipulate the other parameters.

There are a few things we could try in this case:

1. Change our **uid** to another user's **uid**, such that we can take over their accounts
2. Change another user's details, which may allow us to perform several web attacks
3. Create new users with arbitrary details, or delete existing users
4. Change our role to a more privileged role (e.g. **admin**) to be able to perform more actions

Let's start by changing our **uid** to another user's **uid** (e.g. **"uid": 2**). However, any number we set other than our own **uid** gets us a response of **uid mismatch**:

Pretty	Raw	Hex	\n	≡	Pretty	Raw	Hex	Render	\n	≡
1	PUT	/profile/api.php/profile/1	HTTP/1.1		1	HTTP/1.1	200	OK		
2	Host:	178.128.160.242:32504			2	Date:				
3	Content-Length:	208			3	Server:	Apache/2.4.41 (Ubuntu)			
4	User-Agent:	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)			4	Content-Length:	12			
5	Content-type:	application/json			5	Connection:	close			
6	Accept:	/*/*			6	Content-Type:	text/html; charset=UTF-8			
7	Origin:	http://178.128.160.242:32504			7					
8	Referer:	http://178.128.160.242:32504/profile/index.php			8	uid mismatch				
9	Accept-Encoding:	gzip, deflate								
10	Accept-Language:	en-US,en;q=0.9								
11	Cookie:	role=employee								
12	Connection:	close								
13										
14	{									
	"uid":2,									
	"uuid":"40f5888b67c748df7efba008e7c2f9d2",									
	"role":"employee",									
	"full_name":"Amy Lindon",									
	"email":"a_lindon@employees.htb",									
	"about":"A Release is like a boat. 80% of the holes plugged is not good enough."									
	}									

The web application appears to be comparing the request's `uid` to the API endpoint (`/1`). This means that a form of access control on the back-end prevents us from arbitrarily changing some JSON parameters, which might be necessary to prevent the web application from crashing or returning errors.

Perhaps we can try changing another user's details. We'll change the API endpoint to `/profile/api.php/profile/2`, and change `"uid": 2` to avoid the previous `uid mismatch`:

Request

PrettyRawHex\n≡

```
1 PUT /profile/api.php/profile/2 HTTP/1.1
2 Host: 178.128.160.242:32504
3 Content-Length: 208
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
5 Content-type: application/json
6 Accept: */*
7 Origin: http://178.128.160.242:32504
8 Referer: http://178.128.160.242:32504/profile/index.php
9 Accept-Encoding: gzip, deflate
10 Accept-Language: en-US,en;q=0.9
11 Cookie: role=employee
12 Connection: close
13
14 {
  "uid":2,
  "uuid":"40f5888b67c748df7efba008e7c2f9d2",
  "role":"employee",
  "full_name":"Amy Lindon",
  "email":"a_lindon@employees.htb",
  "about":"A Release is like a boat. 80% of the holes plugged is not good enough."
}
```

Response

PrettyRawHexRender\n≡

```
1 HTTP/1.1 200 OK
2 Date:
3 Server: Apache/2.4.41 (Ubuntu)
4 Content-Length: 13
5 Connection: close
6 Content-Type: text/html; charset=UTF-8
7
8 uid mismatch
```

As we can see, this time, we get an error message saying `uuid mismatch`. The web application appears to be checking if the `uuid` value we are sending matches the user's `uuid`. Since we are sending our own `uuid`, our request is failing. This appears to be another form of access control to prevent users from changing another user's details.

Next, let's see if we can create a new user with a `POST` request to the API endpoint. We can change the request method to `POST`, change the `uid` to a new `uid`, and send the request to the API endpoint of the new `uid`:

Request

Pretty

Raw

Hex

\n

⋮

1

POST /profile/api.php/profile/50 HTTP/1.1

2

Host: 178.128.160.242:32504

3

Content-Length: 129

4

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)

5

Content-type: application/json

6

Accept: */*

7

Origin: http://178.128.160.242:32504

8

Referer: http://178.128.160.242:32504/profile/index.php

9

Accept-Encoding: gzip, deflate

10

Accept-Language: en-US,en;q=0.9

11

Cookie: role=employee

12

Connection: close

13

14

{

"uid":50,

"uuid":"40f5888b67c748df7efba008e7c2f9d2",

"role":"employee",

"full_name":"Test",

"email":"test@employees.htb",

"about": ""

}

Response

Pretty

Raw

Hex

Render

\n

⋮

1

HTTP/1.1 200 OK

2

Date:

3

Server: Apache/2.4.41 (Ubuntu)

4

Content-Length: 41

5

Connection: close

6

Content-Type: text/html; charset=UTF-8

7

8

Creating new employees is for admins only

We get an error message saying `Creating new employees is for admins only`. The same thing happens when we send a `Delete` request, as we get `Deleting employees is for admins only`. The web application might be checking our authorization through the

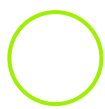
Finally, let's try to change our `role` to `admin/administrator` to gain higher privileges. Unfortunately, without knowing a valid `role` name, we get `Invalid role` in the HTTP response, and our `role` does not update:

Request	Response
<pre> 1 PUT /profile/api.php/profile/1 HTTP/1.1 2 Host: 178.128.160.242:32504 3 Content-Length: 205 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) 5 Content-type: application/json 6 Accept: */* 7 Origin: http://178.128.160.242:32504 8 Referer: http://178.128.160.242:32504/profile/index.php 9 Accept-Encoding: gzip, deflate 10 Accept-Language: en-US,en;q=0.9 11 Cookie: role=employee 12 Connection: close 13 14 { "uid":1, "uuid":"40f5888b67c748df7efba008e7c2f9d2", "role":"admin", "full_name":"Amy Lindon", "email":"a_lindon@employees.htb", "about":"A Release is like a boat. 80% of the holes plugged is not good enough." }</pre>	<pre> 1 HTTP/1.1 200 OK 2 Date: 3 Server: Apache/2.4.41 (Ubuntu) 4 Content-Length: 12 5 Connection: close 6 Content-Type: text/html; charset=UTF-8 7 8 Invalid role</pre>

So, all of our attempts appear to have failed. We cannot create or delete users as we cannot change our `role`. We cannot change our own `uid`, as there are preventive measures on the back-end that we cannot control, nor can we change another user's details for the same reason. So, is the web application secure against IDOR attacks?.

So far, we have only been testing the IDOR Insecure Function Calls. However, we have not tested the API's `GET` request for IDOR Information Disclosure Vulnerabilities. If there was no robust access control system in place, we might be able to read other users' details, which may help us with the previous attacks we attempted.

Try to test the API against IDOR Information Disclosure vulnerabilities by attempting to get other users' details with `GET` requests. If the API is vulnerable, we may be able to leak other users' details and then use this information to complete our IDOR attacks on the function calls.



Connect to Pwnbox

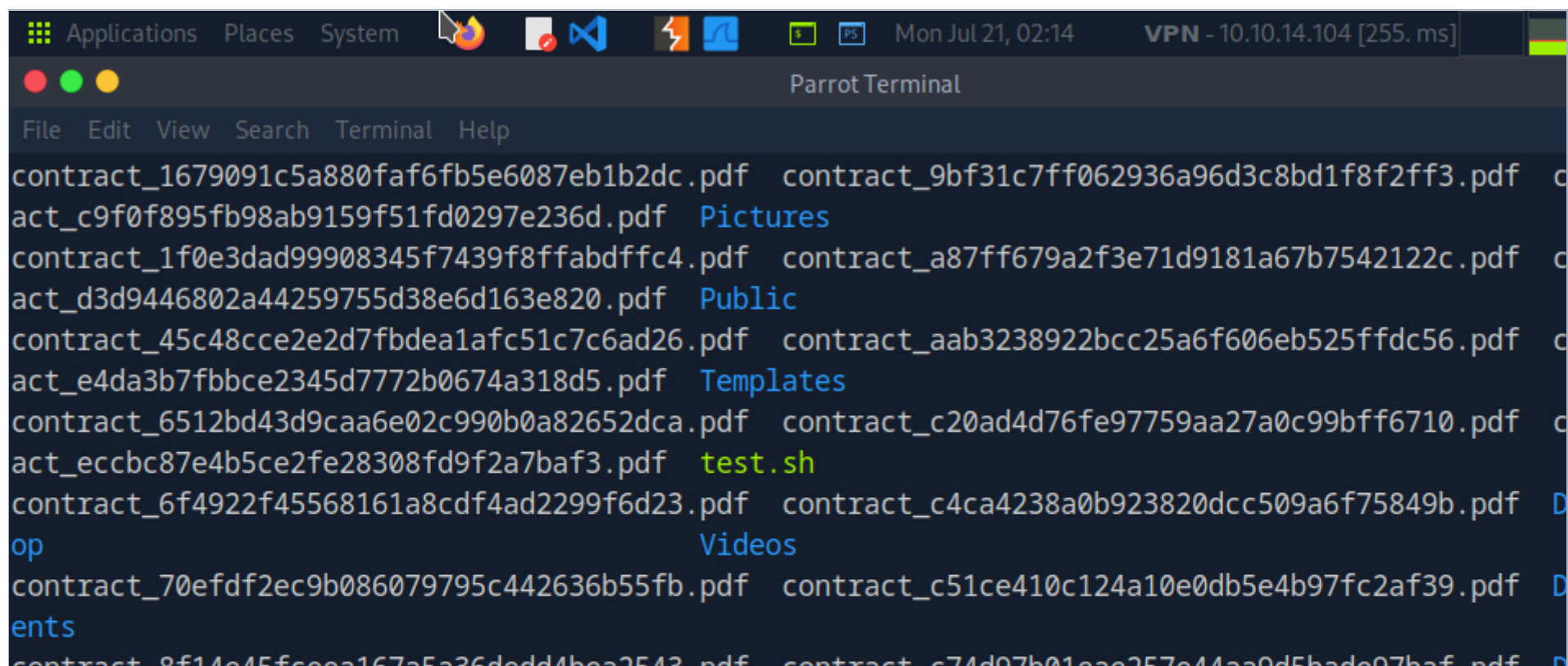
Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

293ms

Terminate Pwnbox to switch location



Integrated Terminal

```
[eu-academy-3]-[10.10.14.104]-[htb-ac-819475@htb-mew08x7bnw]-[~]  
[*]$ cat *.pdf  
HTB{h45h1n6_1d5_w0n7_570p_m3}  
[eu-academy-3]-[10.10.14.104]-[htb-ac-819475@htb-mew08x7bnw]-[~]  
[*]$
```

Full Screen

Terminate

Reset

Life Left: 95m

+

Connected to htb-mew08x7bnw:1 (htb-ac-819475)



4. Questions

Cheat Sheet

Answer the question(s) below to complete this Section and earn cubes!

Target(s): 94.237.121.185:48859

Life Left: 64 minute(s)

+ 1 Try to read the details of the user with 'uid=5'. What is their 'uuid' value?

Submit your answer here...

+10 Streak pts

Submit

Hint

Show Solution

Previous

Next

Cheat Sheet

? Go to Questions

5. Table of Contents

Introduction to Web Attacks

✓

6. HTTP Verb Tampering

Intro to HTTP Verb Tampering

✓

Bypassing Basic Authentication

✓

Bypassing Security Filters

✓

Verb Tampering Prevention

✓

6. Insecure Direct Object References (IDOR)

Intro to IDOR

✓

Identifying IDORs

✓

Mass IDOR Enumeration

✓

Bypassing Encoded References

✓

IDOR in Insecure APIs

Chaining IDOR Vulnerabilities

Integrated Terminal

6. XML External Entity (XXE) Injection

Intro to XXE

Local File Disclosure

Advanced File Disclosure

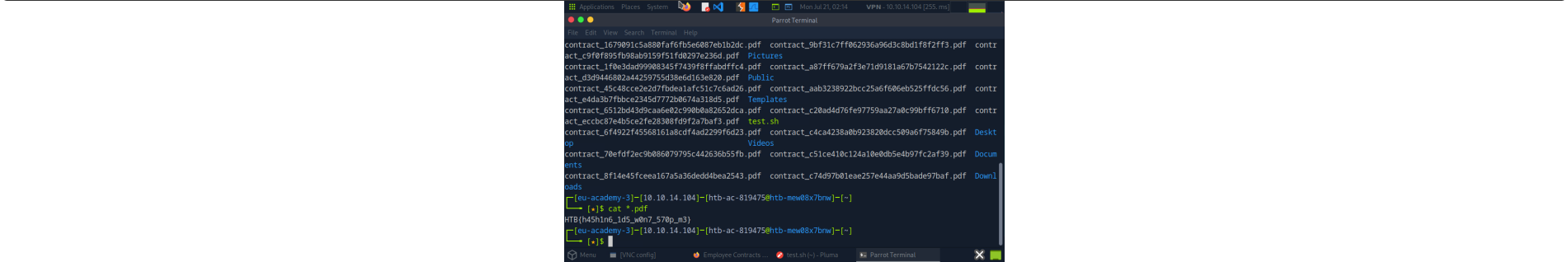
Blind Data Exfiltration

XXE Prevention

6. Skills Assessment

Web Attacks - Skills Assessment

5. My Workstation



Interact

Terminate

Reset

Life Left: 95m

